Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский университет
“Высшая школа экономики”»**

Факультет компьютерных наук
Департамент программной инженерии

Отчёт по индивидуальному заданию № 3

по дисциплине «Архитектура компьютера и операционные системы»

Вариант 14. «Задача об умных цветах»

Имитационное моделирование с помощью взаимодействующих процессов

Выполнила: Гончарова Екатерина
Группа: БПИ-253

Москва, 2026

Содержание

1 Постановка задачи **2**

2 Модель и входные параметры **2**
2.1 Сущности модели 2
2.2 Параметры запуска 2

3 Общая структура программы **3**

4 Вариант 1. Неименованные семафоры и разделяемая память **3**
4.1 Создание процессов 3
4.2 Синхронизация 3

5 Вариант 2. Именованные семафоры и разделяемая память **3**

6 Вариант 3. Независимые процессы и разделяемая память **4**
6.1 Запуск 4
6.2 Обмен данными 4

7 Вариант 4. Независимые процессы и очереди сообщений **4**
7.1 Архитектура 4
7.2 Семафоры и ограничения 4

8 Пользовательский интерфейс и запуск **5**

9 Проверки и результаты работы **5**
9.1 Набор сценариев 5
9.2 Проверка интерактивного запуска 6
9.3 Результаты тестирования 6
9.4 Пример результата работы 6

10 Особенности и ограничения реализации **6**

11 Заключение **7**

1 Постановка задачи

Требуется разработать консольную программу для Linux, моделирующую работу умных цветов и садовников. Цветы постепенно теряют влагу и при достижении заданного порога отправляют заявку на полив. Садовники выбирают заявки, добираются до соответствующего цветка и выполняют полив. Одну заявку может обслуживать только один садовник. Если цветок высох до критического уровня, он погибает; повторный полив в течение защитного интервала также может привести к гибели. Ход моделирования отображается в консоли и записывается в журнал.

В третьем задании участники моделируются **процессами**, а не потоками. Процессы имеют независимые адресные пространства, поэтому обмен состоянием модели требует специальных средств межпроцессного взаимодействия (IPC). Подготовлены четыре варианта с разными средствами синхронизации и передачи данных.

2 Модель и входные параметры

2.1 Сущности модели

`Flower` содержит идентификатор цветка, влажность, скорость высыхания, текущее состояние, данные о заявке, оставшееся время полива, восстановления и защиты, а также число завершённых поливов. `Gardener` содержит идентификатор, занятость, индекс обслуживаемого цветка, объём оставшейся работы и скорость. `Settings` хранит конфигурацию всего сада.

Используются состояния `NORMAL`, `WILTING`, `WATERING`, `RECOVERING`, `DRY` и `DEAD_OVERWATERED`. Начальная влажность каждого цветка составляет 100. После достижения порога заявки создаётся запрос на полив; если влажность достигла порога засыхания, заявка снимается и цветок перестаёт обслуживаться.

2.2 Параметры запуска

Параметр	Значение и ограничение
Число цветов	От 1 до 100
Скорости высыхания	По одному положительному целому числу на цветок
Порог заявки	Целое число от 1 до 100
Порог засыхания	Целое число от 0 до 99; строго меньше порога заявки
Длительность полива	Положительное целое число тактов
Восстановление	Неотрицательное число тактов
Защитный интервал	Неотрицательное число тактов
Число садовников	От 1 до 10
Скорость садовника	Положительное целое число
Правило выбора	`first` либо `driest`

Например, при пороге заявки 30 и пороге засыхания 10 цветок запрашивает полив при снижении влажности до 30, а при дальнейшем снижении до 10 погибает. Правило `first` выбирает первую доступную заявку; `driest` — заявку от цветка с минимальной влажностью. Время модели ограничено 50 тактами. В реализации используется задержка 25 мс на такт; это задержка отображения и выполнения, а не реальное время роста растений.

3 Общая структура программы

Основная модель разделена на небольшие файлы: `flower_logic.c` обновляет отдельный цветок, `gardener_logic.c` управляет поливом, `request_logic.c` выбирает заявки, `model.c` инициализирует структуры, `input.c` и `config_parser.c` читают параметры, `statistics.c` рассчитывает итоги, `logger.c` записывает события. В каждой реализации изменяются прежде всего `simulation.c`, функции процессов и IPC-модуль. Такой подход позволяет сравнивать механизмы взаимодействия без изменения правил поведения цветов.

В первых двух вариантах запускается `smart_flowers`, в третьем и четвёртом — три исполняемых файла: `garden`, `flower`, `gardener`. Главный процесс или контроллер отвечает за создание ресурсов, контроль времени, остановку и вывод итогов.

4 Вариант 1. Неименованные семафоры и разделяемая память

4.1 Создание процессов

Основная программа считывает `Settings`, создаёт POSIX-объект разделяемой памяти через `shm_open()`, задаёт его размер с помощью `ftruncate()` и отображает в адресное пространство вызовом `mmap()`. Затем родитель создаёт дочерние процессы через `fork()`: один процесс на каждый цветок и каждый садовник.

Поскольку после `fork()` обычные локальные переменные процессов независимы, состояния цветов, заявки и признаки остановки размещаются в `SharedModel`, доступной всем процессам.

4.2 Синхронизация

Внутри разделяемой памяти инициализируются неименованные POSIX-семафоры через `sem_init()` с параметром `pshared=1`. Семафор модели обеспечивает взаимное исключение при проверке и изменении заявок. Отдельные семафоры уведомляют садовников о появлении работы. Проверка свободной заявки и её закрепление выполняются в одной критической секции; два садовника не могут одновременно принять одну заявку.

При завершении родитель останавливает детей, дожидается их через `waitpid()`, уничтожает семафоры и удаляет shared memory. Обработка `SIGINT` позволяет выполнить тот же порядок очистки при `Ctrl+C`.

5 Вариант 2. Именованные семафоры и разделяемая память

Процессы запускаются тем же способом — родителем через `fork()`; память также создаётся через `shm_open()` и `mmap()`. Отличается тип семафоров. Вместо `sem_init()` используются `sem_open()` и уникальные имена семафоров, связанные с запуском программы.

Процесс получает доступ к именованному семафору по его имени. Для завершения вызываются `sem_close()` и `sem_unlink()`. Это важно: закрытие дескриптора само по себе не удаляет имя объекта из системы. Модель цветов и правила назначения заявок в первом и втором вариантах одинаковы. Разница заключается именно в организации семафоров.

6 Вариант 3. Независимые процессы и разделяемая память

6.1 Запуск

Третий вариант не создаёт рабочие процессы через `fork()` внутри контроллера. Вместо этого отдельные исполняемые файлы `garden`, `flower` и `gardener` могут запускаться из разных терминалов. Контроллер `garden` создаёт именованные семафоры и разделяемую память, а работники подключаются к ним по имени сессии. Каждый работник получает роль и номер, начиная с 1.

Контроллер ждёт регистрации ожидаемого числа цветов и садовников. Для предотвращения повторного подключения одного работника проверяются идентификаторы и занятые позиции. Если не все процессы зарегистрировались в течение 15 секунд, запуск завершается с ошибкой и очисткой доступных ресурсов.

6.2 Обмен данными

Все участники читают и изменяют одну общую структуру в POSIX shared memory. Именованный семафор модели ограничивает доступ к этой структуре, чтобы состояние заявки и назначение садовника изменялись согласованно. Уведомления о новых заявках передаются через отдельные именованные семафоры садовников.

Контроллер не является родителем работников и поэтому не может обычным `waitpid()` дождаться этих независимых процессов. Он проверяет их регистрацию и доступность, устанавливает общий признак остановки и удаляет собственные IPC-ресурсы при завершении сессии.

7 Вариант 4. Независимые процессы и очереди сообщений

7.1 Архитектура

Четвёртый вариант также состоит из независимо запускаемых программ `garden`, `flower` и `gardener`, но **состояние модели не передаётся через общую память**. Контроллер хранит основной экземпляр модели в собственной памяти. Для обмена с работниками используются POSIX-очереди сообщений, создаваемые через `mq_open()`.

Для сессии создаются очередь событий, через которую работники передают регистрацию и результаты, и отдельные очереди заданий для работников. Контроллер передаёт снимок состояния, роль и номер такта. Получив сообщение, процесс цветка или садовника применяет свою часть логики и отправляет результат обратно. Контроллер проверяет номер отправителя и такта и вносит изменение в основную модель.

7.2 Семафоры и ограничения

Именованный семафор используется для уведомления контроллера о поступившем событии. Дополнительные именованные семафоры не допускают двойной регистрации одного номера работника. Последовательное применение результатов садовников предотвращает конфликтующие изменения состояния заявки.

В очереди каждого работника достаточно одного ожидающего задания: следующий шаг отправляется только после обработки предыдущего. Это также уменьшает объём па-

мяти IPC и позволяет запускать конфигурации с большим количеством рабочих процессов без избыточного резервирования очередей. При завершении контроллер отправляет сообщения остановки и удаляет имена очередей через `mq_unlink()`, а семафоров — через `sem_unlink()`.

8 Пользовательский интерфейс и запуск

Все команды выполняются в Linux из каталога соответствующего варианта. Компиляция проводится с помощью `make`, автоматические сценарии — с помощью `make check`. Для первых двух вариантов поддерживаются аргумент с файлом конфигурации и ручной ввод параметров без файла:

```
make
./build/smart_flowers tests/balanced.cfg
./build/smart_flowers
```

В вариантах 3 и 4 можно запустить контроллер с файлом и именем сессии или с одним именем для ручного ввода:

```
make
./build/garden tests/one_flower.cfg demo
./build/garden demo
```

Во втором случае контроллер последовательно запрашивает число цветов, индивидуальные скорости высыхания, пороги, длительности, число садовников, скорость и правило выбора заявки. Некорректное число можно ввести повторно. После настройки остальные процессы подключаются из других терминалов:

```
./build/flower demo 1
./build/gardener demo 1
```

Для конфигураций с несколькими участниками требуется запустить экземпляр программы для каждого номера. Сценарий `scripts/run_demo.sh` автоматически определяет количество работников из конфигурационного файла и запускает их в отдельных фоновых процессах:

```
bash scripts/run_demo.sh tests/balanced.cfg
```

При таком запуске имя сессии выбирается скриптом автоматически. Для ручного запуска нужно задать своё имя: от 1 до 24 латинских букв, цифр, символов подчёркивания или дефисов. Все ключевые события видны в консоли. Результаты сохраняются в `simulation.log` либо `simulation_`*`session`*`.log` в независимых вариантах. В штатной ситуации симуляция завершается после 50 тактов или когда все цветы стали неактивны. Нажатие `Ctrl+C` в терминале главной программы останавливает выполнение раньше.

9 Проверки и результаты работы

9.1 Набор сценариев

Для вариантов предусмотрены конфигурации, проверяющие работу нескольких садовников, стратегию `driest`, быстрое высыхание, засыхание в начале работы, длительное восстановление, отсутствие защитного интервала, перелив, одного работника, большое число процессов и крайние допустимые значения порогов. Всего используется 13 файлов конфигурации.

Дополнительно тестируются некорректные аргументы, недопустимые значения параметров, прерывание по `Ctrl+C`, корректное завершение работников и очистка IPC-ресурсов. В третьем и четвёртом вариантах отдельно проверяются независимое подключение, повторная регистрация и недопустимые номера процессов.

9.2 Проверка интерактивного запуска

Для вариантов 3 и 4 был отдельно проверен ручной ввод с одним цветком и одним садовником. Сначала в качестве порога засыхания было введено число, превышающее порог заявки; программа сообщила об ошибке и запросила значение повторно. После исправления параметра, запуска `flower` и `gardener` контроллер дождался обоих участников, выполнил симуляцию и вывел итоговую статистику. Обе проверки завершились успешно.

9.3 Результаты тестирования

Для каждого варианта использовались 13 конфигурационных файлов с различными входными параметрами. Проверки запускались командой `make check`. Результаты приведены в таблице.

Реализация	Результат
Вариант 1 (неименованные семафоры)	13 сценариев пройдены
Вариант 2 (именованные семафоры)	13 сценариев пройдены
Вариант 3 (независимые процессы, разделяемая память)	13 сценариев пройдены
Вариант 4 (независимые процессы, очереди сообщений)	13 сценариев пройдены

Все четыре реализации успешно прошли предусмотренные тестовые сценарии.

Итоговый вывод программы содержит последний обработанный такт, число живых и погибших цветов, общее число поливов, а также состояние и влажность каждого цветка. Журнал позволяет проследить создание заявки, её принятие, движение садовника, завершение полива и переходы между состояниями. Конкретные числовые результаты зависят от выбранной конфигурации и порядка выполнения процессов, поэтому для воспроизводимого наблюдения целесообразно использовать конфигурации из каталога `tests/`.

9.4 Пример результата работы

Для примера, при запуске четвёртой реализации с `tests/one_flower.cfg` один из запусков завершился на 50-м такте со следующими показателями: один живой цветок, ноль засохших, ноль погибших от перелива и семь завершённых поливов. В первых трёх вариантах число операций и промежуточные состояния могут отличаться, поскольку процессы обновляют модель асинхронно.

10 Особенности и ограничения реализации

В первых трёх вариантах общая разделяемая структура защищается семафором. Такой способ прост и позволяет использовать общий код модели, но при большом числе работников ограничивает параллельное изменение данных. В четвёртом варианте основные из-

менения последовательно применяет контроллер. Это устраняет конфликтующие записи, но каждый шаг обслуживания требует передачи сообщений и ожидания ответа.

Ограничение на число POSIX-очередей и объём выделяемой им памяти зависит от системы Linux. В четвёртом варианте персональные очереди имеют вместимость в одно задание, что соответствует последовательному протоколу обмена. При жёстком принудительном завершении `SIGKILL` нельзя гарантировать удаление всех именованных объектов: этот сигнал не обрабатывается программой. При нормальном завершении и `Ctrl+C` выполняются предусмотренные операции очистки.

11 Заключение

В рамках задания реализованы четыре способа организации взаимодействия процессов. Первые два варианта показывают различие между неименованными и именованными POSIX-семафорами при использовании общего родителя. Третий позволяет запускать участников независимо, подключая их к общей памяти по имени сессии. Четвёртый использует обмен сообщениями без разделяемой памяти модели. Поведение цветов и садовников во всех реализациях построено на одной предметной модели, поэтому различия относятся прежде всего к организации процессов, передаче данных и синхронизации.